TP N°1 – Sécurité et Cryptographie Appliquée

Algorithme d'Euclide étendu, Inversion modulaire et Chiffrement affine

Yacine Maamar

February 2026

1 Implémentation arithmétique – Classe `BigIntMath`

La classe `BigIntMath` implémente l'algorithme d'Euclide étendu (`extendedGCD`) ainsi que le calcul de l'inverse modulaire (`modInverse`). La méthode `extendedGCD(a, b)` retourne un triplet (d, u, v) tel que $d = \gcd(a, b)$ et $au + bv = d$. La méthode `modInverse(a, n)` vérifie la condition d'inversibilité, retourne l'inverse dans $\{0, \ldots, n-1\}$, et signale explicitement l'absence d'inverse lorsque $\gcd(a, n) \neq 1$.

Listing 1 – Classe BigIntMath : Euclide étendu et inversion modulaire

```
class BigIntMath:
    """
    Classe fournissant des operations arithmetiques sur les grands entiers,
    notamment l'algorithme d'Euclide etendu et l'inversion modulaire.
    """

    @staticmethod
    def extendedGCD(a, b):
        """
        Retourne (d, u, v) tel que d = gcd(a, b) et a*u + b*v = d.
        """
        # Initialisation selon l'algorithme de la fiche
        r, r_prime = a, b
        u, v = 1, 0
        u_prime, v_prime = 0, 1

        while r_prime != 0:
            q = r // r_prime
            r, u, v, r_prime, u_prime, v_prime = (
                r_prime,
                u_prime,
                v_prime,
                r - q * r_prime,
                u - q * u_prime,
                v - q * v_prime
            )

        # r = gcd(a, b), u et v sont les coefficients de Bezout
        return (r, u, v)

    @staticmethod
    def modInverse(a, n):
        """
        Retourne l'inverse de a modulo n dans {0, ..., n-1}.
        Leve une ValueError si gcd(a, n) != 1.
        """
        d, u, _ = BigIntMath.extendedGCD(a, n)
```

```
    # Verification de la condition d'inversibilite
    if d != 1:
        raise ValueError(
            f"L'inverse de {a} modulo {n} n'existe pas : "
            f"gcd({a}, {n}) = {d} != 1."
        )

    # Ramener u dans l'intervalle {0, ..., n-1}
    return u % n
```

2 Extension – Chiffrement affine – Classe `AffineCipher`

La classe `AffineCipher` implémente le chiffrement affine défini par :

$$E_{a,b}(m) = (am + b) \bmod n \qquad et \qquad D_{a,b}(c) = a^{-1}(c - b) \bmod n$$

Le déchiffrement utilise exclusivement `modInverse` développé en section 1. La vérification expérimentale $D(E(m)) = m$ est incluse dans le bloc principal.

Listing 2 – Classe AffineCipher : chiffrement et déchiffrement affine

```
class AffineCipher:

    def __init__(self, a, b, n=26):
        self.n = n
        self.b = b % n
        self.a = a % n
        # Calcul de l'inverse de a via modInverse
        self.a_inv = BigIntMath.modInverse(self.a, self.n)

    def encrypt(self, m):
        """ E(m) = (a * m + b) mod n """
        return (self.a * m + self.b) % self.n

    def decrypt(self, c):
        """ D(c) = a^{-1} * (c - b) mod n """
        return (self.a_inv * (c - self.b)) % self.n

    def encrypt_text(self, plaintext):
        """ Chiffre une chaine de caracteres (lettres minuscules). """
        result = []
        for ch in plaintext.lower():
            if ch.isalpha():
                m = ord(ch) - ord('a')
                c = self.encrypt(m)
                result.append(chr(c + ord('a')))
            else:
                result.append(ch)
        return ''.join(result)

    def decrypt_text(self, ciphertext):
        """ Dechiffre une chaine de caracteres (lettres minuscules). """
        result = []
        for ch in ciphertext.lower():
            if ch.isalpha():
                c = ord(ch) - ord('a')
                m = self.decrypt(c)
                result.append(chr(m + ord('a')))
            else:
                result.append(ch)
        return ''.join(result)

if __name__ == "__main__":
    a, b, n = 7, 3, 26
    cipher = AffineCipher(a, b, n)

    # Verification sur tous les entiers de {0, ..., 25}
    ok = all(cipher.decrypt(cipher.encrypt(m)) == m for m in range(n))
    print(f"D(E(m)) = m pour tout m in {{0,...,25}} : {ok}")

    # Exemple sur un texte
    plaintext = "cryptographie"
    ciphertext = cipher.encrypt_text(plaintext)
    decrypted  = cipher.decrypt_text(ciphertext)
    print(f"Clair    : {plaintext}")
    print(f"Chiffre  : {ciphertext}")
    print(f"Dechiffre: {decrypted}")
    print(f"D(E(m))=m: {plaintext == decrypted}")
```

0.8/1